


# OPERATING SYSTEMS
# UE25CS242A

## Process Creation and Termination, System calls for Process Management

**Dr. Anitha K**
Department of Computer Science & Engineering

**Acknowledgement, Prof. Pavan A C**

## Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources:




1. "Operating System Concepts", Abraham Silberschatz, Peter Baer Galvin, Greg Gagne 10th Edition, John Wiley & Sons, Global Edition, 2023.
2. "Advanced Programming in the Unix Environment", Richard Stevens and Stephen A Rago, Pearson, 3rd edition, 2017.
3. Some conceptual text and diagram from "Operating Systems, Internals and Design Principles", William Stallings, 9th Edition, Pearson, 2018

# OPERATING SYSTEMS

## Process Creation and Termination , System calls for Process Management

**Dr. Anitha K**
Department of Computer Science and Engineering

# OPERATING SYSTEMS

## Operations on Processes

System must provide mechanisms for:

- process creation
- process termination

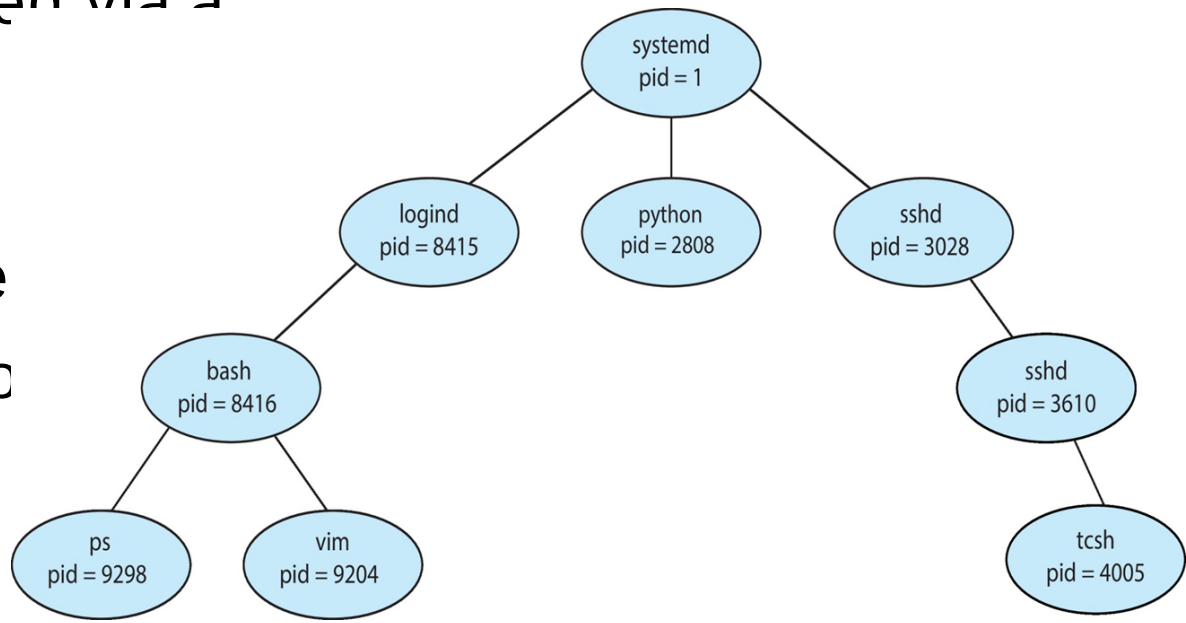
PES UNIVERSITY
CELEBRATING 50 YEARS

# OPERATING SYSTEMS

## Process Creation

- **Parent** process creates **children** processes, which, in turn create other processes, forming a **tree** of processes
- Generally, process identified and managed via a **process identifier** (**pid**)
- Resource sharing options
  - Parent and children share all resource
  - Children share subset of parent's reso
  - Parent and child share no resources
- Execution options
  - Parent and children execute concurrently
  - Parent waits until children terminate

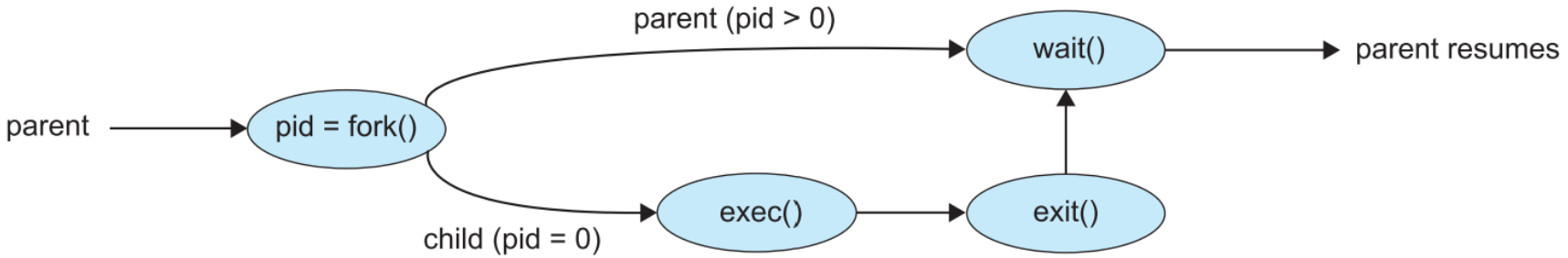
**A tree of processes on a typical Linux system.**




© copyright Silberschatz, Galvin and Gagne, 2013

# Process creation using fork()

- Address space
  - Child duplicate of parent
  - Child has a program loaded into it
- UNIX examples
  - **fork()** system call creates new process
  - **exec()** system call used after a **fork()** to replace the process' memory space with a




**Process creation using the fork() system call.**

# C Program forking Separate Process




```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
       fprintf(stderr, "Fork Failed");
       return 1;
    }
    else if (pid == 0) { /* child process */
       execlp("/bin/ls","ls",NULL);
    }
    else { /* parent process */
       /* parent will wait for the child to complete */
       wait(NULL);
       printf("Child Complete");
    }

    return 0;
}
```

# Process Termination

- Process executes last statement and then asks the operating system to delete it using the **exit()** system call.
  - Returns status data from child to parent (via **wait()**)
  - Process' resources are deallocated by operating system
- Parent may terminate the execution of children processes using the **abort()** system call. Some reasons for doing so:
  - Child has exceeded allocated resources
  - Task assigned to child is no longer required
  - The parent is exiting and the operating systems does not allow a child to continue if its parent terminates

# OPERATING SYSTEMS

## Process Termination




- Some operating systems do not allow child to exist if its parent has terminated. If a process terminates, then all its children must also be terminated.
  - **cascading termination.** All children, grandchildren, etc. are terminated.
  - The termination is initiated by the operating system.
- The parent process may wait for termination of a child process by using the **wait()** system call**.** The call returns status information and the pid of the terminated process
  - **pid = wait(&status);**
- If no parent waiting (did not invoke **wait()**) process is a **zombie**
- If parent terminated without invoking **wait** , process is an **orphan**

## System calls for Process Management- Process Identifiers

- Every process has a unique **process ID**, a non-negative integer
- The process ID is the only well-known identifier of a process that is always unique
- It is often used as a piece of other identifiers, to guarantee uniqueness.
- Although unique, process IDs are reused. As processes terminate, their IDs become candidates for reuse.
- Most UNIX systems implement algorithms to delay reuse, so that newly created processes are assigned IDs different from those used by processes that terminated recently

# fork()

- An existing process can create a new one by calling the **fork** function

```
#include <unistd.h>
pid_t fork(void);
```

- The new process created by fork is called the child process
- Return value in the child is 0
- Return value in the parent is the process ID of the new child
- Return value is -1 on error
- The child is a copy of the parent. The child gets a copy of the parent's data space, heap, and stack

# exit()

- A process can terminate normally in five ways
  1. Executing a return from the main function
  2. Calling the **exit** function.
  3. Calling the _exit or _Exit function.
  4. Executing a return from the start routine of the last thread in the process.
  5. Calling the pthread_exit function from the last thread in the process

A process can terminate abnormally in three ways

  1. Calling abort
  2. When the process receives certain signals
  3. The last thread responds to a cancellation request

- Regardless of how a process terminates, the same code in the kernel is eventually executed.
- This kernel code closes all the open descriptors for the process, releases the memory

# wait() and waitpid()

- A process that calls wait or waitpid can
  - Block, if all of its children are still running
  - Return immediately with the termination status of a child, if a child has terminated and is waiting for its termination status to be fetched
  - Return immediately with an error, if it doesn't have any child processes
- If the process is calling wait because it received the SIGCHLD signal, wait will return immediately. But if we call it at any random point in time, it can block.

```
#include <sys/wait.h>

pid_t wait(int *statloc);

pid_t waitpid(pid_t pid, int *statloc, int options);
```

Both return: process ID on success, 0 if state hasn't changed or −1 on failure

# waitid()

- waitid() allows a process to specify which children to wait for.
- Instead of encoding this information in a single argument combined with the process ID or process group ID, two separate arguments are used

```
#include <sys/wait.h>
int waitid(idtype_t idtype, id_t id, siginfo_t *infop, int options);
```

- Returns: 0 if OK, −1 on error

# exec()

- When a process calls one of the exec functions, that process is completely replaced by the new program, and the new program starts executing at its main function.
- The process ID does not change across an exec, because a new process is not created;
- exec merely replaces the current process — its text, data, heap, and stack segments

```
—\#include <unistd.h>

    int execl(const char *pathname, const char *arg0, ... /* (char *)0 */ );

    int execv(const char *pathname, char *const argv[]);

    int execle(const char *pathname, const char *arg0, ...
                /* (char *)0, char *const envp[] */ );

    int execve(const char *pathname, char *const argv[], char *const envp[]);

    int execlp(const char *filename, const char *arg0, ... /* (char *)0 */ );

    int execvp(const char *filename, char *const argv[]);

    int fexecve(int fd, char *const argv[], char *const envp[]);
```

# getpid() and getppid()

```
#include <unistd.h>
pid_t getpid(void);


pid_t getppid(void);
```

- **getpid**() returns the process ID (PID) of the calling process.

- **getppid**() returns the process ID of the parent of the calling process.
- This will be either the ID of the process that created this process using **fork**() or if that process has already terminated, the ID of the process to which this process has been re-parented

# OPERATING SYSTEMS

## Race condition

- A race condition occurs when multiple processes are trying to do something with shared data and the final outcome depends on the order in which the processes run.
- The fork function is a lively breeding ground for race conditions, if the logic after the fork either explicitly or implicitly depends on whether the parent or child runs first after the fork.
- In general, which process runs first cannot be predicted
- A process that wants to wait for a child to terminate must call one of the wait

```
while (getppid() != 1)
    sleep(1);
```

- If a process wants to wait for its parent to terminate, a loop of the following form could be used:

- The problem with this type of loop, called polling, is that it wastes CPU time, as the caller is awakened every second to test the condition

# I/O Request Life Cycle (I/O-Bound Process)

- Requests I/O operations frequently (read/write).
- Moves to waiting state while I/O is performed.
- CPU is allocated to other processes during this time.
- Device sends interrupt after completing I/O.
- Process resumes briefly and again requests I/O.




# THANK YOU

**Dr. Anitha K**
Department of Computer Science & Engineeri

**anithak@pes.edu**